

使用 NPM 在 React 中添加 Bootstrap

打開一個新的終端窗口，引導到專案的文件夾，然後執行以下命令：

```
$ npm install bootstrap --save
```

--save 選項在 *package.json* 文件中添加一個資料項

打開 *src/index.js* 文件並添加以下程式碼：

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

完整程式碼 *index.js*：

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
import reportWebVitals from './reportWebVitals';
import 'bootstrap/dist/css/bootstrap.min.css';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

// If you want to start measuring performance in your app, pass a function
// to log results (for example: reportWebVitals(console.log))
// or send to an analytics endpoint. Learn more: https://bit.ly/CRA-vitals
reportWebVitals();
```

EmployeeService - 使用 CRUD REST API

為了在 React 應用程式中執行 REST API，我們將使用 Axios HTTP 程式庫。

讓我們使用以下 NPM 命令安裝 Axios：

```
npm add axios --save
```

接下來，我們建一個 *EmployeeService.js* 文件並向其中添加以下程式碼：

```
import axios from 'axios'

const EMPLOYEE_BASE_REST_API_URL = 'http://localhost:8080/api/v1/employees';

export const listEmployees = () => {
  return axios.get(EMPLOYEE_BASE_REST_API_URL)
};

export const createEmployee = (employee) => {
  return axios.post(EMPLOYEE_BASE_REST_API_URL, employee)
}

export const getEmployeeById = (employeeId) => {
  return axios.get(EMPLOYEE_BASE_REST_API_URL + '/' + employeeId);
}

export const updateEmployee = (employeeId, employee) => {
  return axios.put(EMPLOYEE_BASE_REST_API_URL + '/' + employeeId, employee);
}

export const deleteEmployee = (employeeId) => {
  return axios.delete(EMPLOYEE_BASE_REST_API_URL + '/' + employeeId);
}
```

React 列表員工組件

我們將在 `src` 文件夾中建一個名為 `Components` 的新文件夾。然後建一個名為 `ListEmployeeComponent.js` 的新文件。

在此文件中建一個名為 `ListEmployeeComponent` 的 React 函數組件，其中包含以下內容：

```
import React, {useState, useEffect} from 'react'
import {listEmployees} from '../services/EmployeeService'

const ListEmployeeComponent = () => {

  const [employees, setEmployees] = useState([])

  useEffect(() => {
    getAllEmployees();
  }, [])

  const getAllEmployees = () => {
    listEmployees().then((response) => {
      setEmployees(response.data)
      console.log(response.data);
    }).catch(error =>{
      console.log(error);
    })
  }

  return (
    <div className = "container">
      <h2 className = "text-center"> List Employees </h2>
      <table className="table table-bordered table-striped">
        <thead>
          <th> Employee Id </th>
          <th> Employee First Name </th>
          <th> Employee Last Name </th>
          <th> Employee Email Id </th>
        </thead>
      </table>
    </div>
  )
}
```

```

        </thead>
        <tbody>
          {
            employees.map(
              employee =>
                <tr key = {employee.id}>
                  <td> {employee.id} </td>
                  <td> {employee.firstName} </td>
                  <td>{employee.lastName}</td>
                  <td>{employee.email}</td>
                </tr>
            )
          }
        </tbody>
      </table>
    </div>
  )
}

export default ListEmployeeComponent

```

`useState` 是一個 Hook（函數），允許您在函數組件中定義狀態變數。您將初始狀態傳遞給該函數，它返回一個具有當前狀態值（不一定是初始狀態）的變數以及另一個更新該值的函數。

`useEffect` Hook 允許我們在函數組件執行。它不使用類組件中可用的組件生命週期方法。`useEffect` 具有大多數 Web 應用程序需要執行的常見功能，例如：

- 更新 DOM，
- 從服務器 API 獲取和使用數據，
- 設置訂閱等

我們正在使用 ES6 功能，它是一個 *map()* 方法來循環我們的員工列表並建視圖：

```
<tbody>
  {
    employees.map(
      employee =>
        <tr key = {employee.id}>
          <td> {employee.id} </td>
          <td> {employee.firstName} </td>
          <td>{employee.lastName}</td>
          <td>{employee.email}</td>
        </tr>
      )
  }
</tbody>
```

頁頭組件和頁腳組件

讓我們建一個名為 *HeaderComponent.js* 的新文件，並在該文件中使用以下程式碼建一個名為 *HeaderComponent* 的組件：

```
import React from 'react'

const HeaderComponent = () => {
  return (
    <div>
      <header>
        <nav className = "navbar navbar-expand-md navbar-dark bg-dark">
          <div>
            <a href = "https://javaguides.net" className =
"navbar-brand">
              Employee Management Application
            </a>
          </div>
        </nav>
      </header>
    </div>
  )
}
```

```

        </div>
    )
}

export default HeaderComponent

```

讓我們建一個名為 *FooterComponent.js* 的新文件，並在該文件中使用以下程式碼建一個名為 *FooterComponent* 的組件：

```

import React from 'react'

const FooterComponent = () => {
    return (
        <div>
            <footer className = "footer">
                <span className="text-muted">All Rights Reserved 2021
                @JavaGuides</span>
            </footer>
        </div>
    )
}

export default FooterComponent

```

將以下 CSS 代碼添加到 *App.css* 文件中：

```

.footer {
    position: absolute;
    bottom: 0;
    width:100%;
    height: 50px;
    background-color: black;
    text-align: center;
    color: white;
}

```

更新 **App.js** 以使用 **FooterComponent** 和 **Test**

```
import './App.css';
import FooterComponent from './components/FooterComponent';
import HeaderComponent from './components/HeaderComponent';
import ListEmployeeComponent from './components/ListEmployeeComponent';

function App() {
  return (
    <div>
      <HeaderComponent />
      <ListEmployeeComponent />
      <FooterComponent />
    </div>
  );
}

export default App;
```

在 React 應用程序中配置路由

要使用 React Router，首先必須使用 NPM 安裝它：

```
npm install react-router-dom --save
```

檢查 *package.json* 文件中 React Router 的資料項。

接下來，讓我們在 *應用程序* 組件中配置路由：

```
import './App.css';
import {BrowserRouter, Routes, Route} from 'react-router-dom';
import FooterComponent from './components/FooterComponent';
import HeaderComponent from './components/HeaderComponent';
import ListEmployeeComponent from './components/ListEmployeeComponent';

function App() {
  return (
```

```
    <div>
      <BrowserRouter>
        <HeaderComponent />
        <div className= "container">
          <Routes>
            <Route path = "/" element = { <ListEmployeeComponent /> }></Route>
            <Route path = "/employees" element = { <ListEmployeeComponent
/> }></Route>
          </Routes>
        </div>
        <FooterComponent />
      </BrowserRouter>
    </div>
  );
}

export default App;
```

Add and Update Employee Component

我們將實現添加員工和更新員工功能。

我們將使用相同的 **React** 組件來執行添加和更新員工操作。

讓我們建一個名為 **EmployeeComponent.js** 的新文件。在此文件中建一個名為 **EmployeeComponent** 的 **React** 組件，其內容如下：


```
import React, {useState, useEffect} from 'react'
import {useNavigate, useParams } from 'react-router-dom';
import { updateEmployee, createEmployee, getEmployeeById} from
'../services/EmployeeService';

const EmployeeComponent = () => {

  const [firstName, setFirstName] = useState('')
  const [lastName, setLastName] = useState('')
  const [email, setEmail] = useState('')

  const navigate = useNavigate();
  const {id} = useParams();

  const saveOrUpdateEmployee = (e) => {
    e.preventDefault();

    const employee = {firstName, lastName, email}

    console.log(employee);
    if(id){
      updateEmployee(id, employee).then((response) => {
        navigate('/employees')
      }).catch(error => {
        console.log(error)
      })
    }else{
      createEmployee(employee).then((response) =>{

        console.log(response.data)

        navigate('/employees');

      }).catch(error => {
        console.log(error)
      })
    }
  }
}
```

```

    }

}

useEffect(() => {

    if(id){
        getEmployeeById(id).then((response) =>{
            setFirstName(response.data.firstName)
            setLastName(response.data.lastName)
            setEmail(response.data.email)
        }).catch(error => {
            console.log(error)
        })
    }

}, [id])

const pageTitle = () => {

    if(id){
        return <h2 className = "text-center">Update Employee</h2>
    }else{
        return <h2 className = "text-center">Add Employee</h2>
    }
}

return (
    <div>
        <br /><br />
        <div className = "container">
            <div className = "row">
                <div className = "card col-md-6 offset-md-3 offset-md-3">
                    {
                        pageTitle()
                    }
                <div className = "card-body">
                    <form>

```

```

        <div className = "form-group mb-2">
            <label className = "form-label"> First
Name :</label>

            <input
                type = "text"
                placeholder = "Enter first name"
                name = "firstName"
                className = "form-control"
                value = {firstName}
                onChange = {(e) =>
                    setFirstName(e.target.value)}
            >
            </input>
        </div>

        <div className = "form-group mb-2">
            <label className = "form-label"> Last
Name :</label>

            <input
                type = "text"
                placeholder = "Enter last name"
                name = "lastName"
                className = "form-control"
                value = {lastName}
                onChange = {(e) =>
                    setLastName(e.target.value)}
            >
            </input>
        </div>

        <div className = "form-group mb-2">
            <label className = "form-label"> Email
Id :</label>

            <input
                type = "email"
                placeholder = "Enter email Id"
                name = "email"
                className = "form-control"

```

```

        value = {email}
        onChange = {(e) =>
setEmail(e.target.value)}
    >
    </input>
</div>

    <button className = "btn btn-success" onClick =
{(e) => saveOrUpdateEmployee(e)} >Submit </button>
    {/* <Link to="/employees" className="btn
btn-danger"> Cancel </Link> */}
    </form>

</div>
</div>
</div>
</div>
)
}

export default EmployeeComponent

```

我們來理解一下上面的程式碼。

我們使用 `useState()` 在我們的功能組件中建狀態變數：

```

const [firstName, setFirstName] = useState('')
const [lastName, setLastName] = useState('')
const [email, setEmail] = useState('')

```

我們 使用以下代碼行從路線中檢索員工 ID：

```

const {id} = useParams();

```

我們建了 JavaScript 函數來處理 onClick 事件（表單提交）：

```

const saveOrUpdateEmployee = (e) => {

```

```
e.preventDefault();

const employee = {firstName, lastName, email, departmentId}

console.log(employee);
if(id){
  updateEmployee(id, employee).then((response) => {
    navigate('/employees')
  }).catch(error => {
    console.log(error)
  })
}else{
  createEmployee(employee).then((response) =>{

    console.log(response.data)

    navigate('/employees');

  }).catch(error => {
    console.log(error)
  })
}

}
```

導引到列出員工頁面從 `react-router-dom` 庫導入 `useNavigate()` :

```
const navigate = useNavigate();
```

導航到 `/employees` 頁面：

```
navigate('/employees')
```

更新 **EmployeeComponent** 的路由

```
import './App.css';
import {BrowserRouter, Routes, Route } from 'react-router-dom';
import FooterComponent from './components/FooterComponent';
import HeaderComponent from './components/HeaderComponent';
import ListEmployeeComponent from './components/ListEmployeeComponent';
import EmployeeComponent from './components/EmployeeComponent';

function App() {
  return (
    <div>
      <BrowserRouter>
        <HeaderComponent />
        <div className= "container">
          <Routes>
            <Route path = "/" element = { <ListEmployeeComponent /> }></Route>
            <Route path = "/employees" element = { <ListEmployeeComponent
/> }></Route>
            <Route path = "/add-employee" element = { <EmployeeComponent /> }
></Route>
            <Route path = "/edit-employee/:id" element = { <EmployeeComponent
/> }></Route>
          </Routes>
        </div>
        <FooterComponent />
      </BrowserRouter>
    </div>
  );
}

export default App;
```

刪除員工功能 - 前端

1. 添加刪除按鈕以列出員工表：

```
<button className = "btn btn-danger" onClick = {() =>
removeEmployee(employee.id)} style = {{marginLeft:"10px"}}> Delete</button>
```

2. 接下來，建 *removeEmployee* 函數並打印 id：

```
const removeEmployee = (employeeId) => {
  console.log(id);
}
```

3. 接下來，我們來使用 *Axios* 編寫刪除員工 REST API 的程式碼：

```
export const deleteEmployee = (employeeId) => {
  return axios.delete(EMPLOYEE_BASE_REST_API_URL + '/' + employeeId);
}
```

4. 更改 *removeEmployee* 以執行 *deleteEmployee* 方法：

```
const removeEmployee = (employeeId) => {
  deleteEmployee(employeeId).then((response) =>{
    getAllEmployees();

  }).catch(error =>{
    console.log(error);
  })
}
```

具有上述更改的 *ListEmployeeComponent* 的完整程式碼：

```
import React, {useState, useEffect} from 'react'
```

```
import { useNavigate } from 'react-router-dom'
import {listEmployees, deleteEmployee} from '../services/EmployeeService'

const ListEmployeeComponent = () => {

  const [employees, setEmployees] = useState([])

  const navigate = useNavigate()

  useEffect(() => {
    getAllEmployees();
  }, [])

  const getAllEmployees = () => {
    listEmployees().then((response) => {
      setEmployees(response.data)
      console.log(response.data);
    }).catch(error =>{
      console.log(error);
    })
  }

  const removeEmployee = (employeeId) => {
    deleteEmployee(employeeId).then((response) =>{
      getAllEmployees();

    }).catch(error =>{
      console.log(error);
    })
  }

  function addNewEmployee() {
    navigate('/add-employee')
  }

  const updateEmployee = (id) => {
    navigate(`/edit-employee/${id}`)
  }
}
```



```

                                style = {{marginLeft:"10px"}}>
Delete</button>
                                </td>
                                </tr>
                            )
                        }
                    </tbody>
                </table>
            </div>
        )
    }

    export default ListEmployeeComponent

```

運行 React 應用程序

在運行 React App 之前，請確保您的 Spring boot 項目已啟動並正在運行。

使用以下命令啟動項目：

```
npm start
```

一旦，反應應用程序就啟動並運行了。接下來，在瀏覽器中點擊此鏈接：<http://localhost:3000/employees>

React App

localhost:3000/employees

CanvasChannel dashboar...React AppVue 3 CRUD exam...Java 8 Streams Tu...Top Java Multithre...

Employee Management Application

List Employees

Add Employee

Employee Id	Employee First Name	Employee Last Name	Employee Email Id	Action
1	Ramesh	Fadatare	ramesh@gmail.com	Update
2	Tony	Stark	tony@gmail.com	Update
3	Tom	Cruise	tom@gmail.com	Update
4	Umesh	Fadatare	umesh@gmail.com	Update